

Name: Alok Kumar Tarini

Reg_no.: 2141020010

CA-Software
Assignment-1

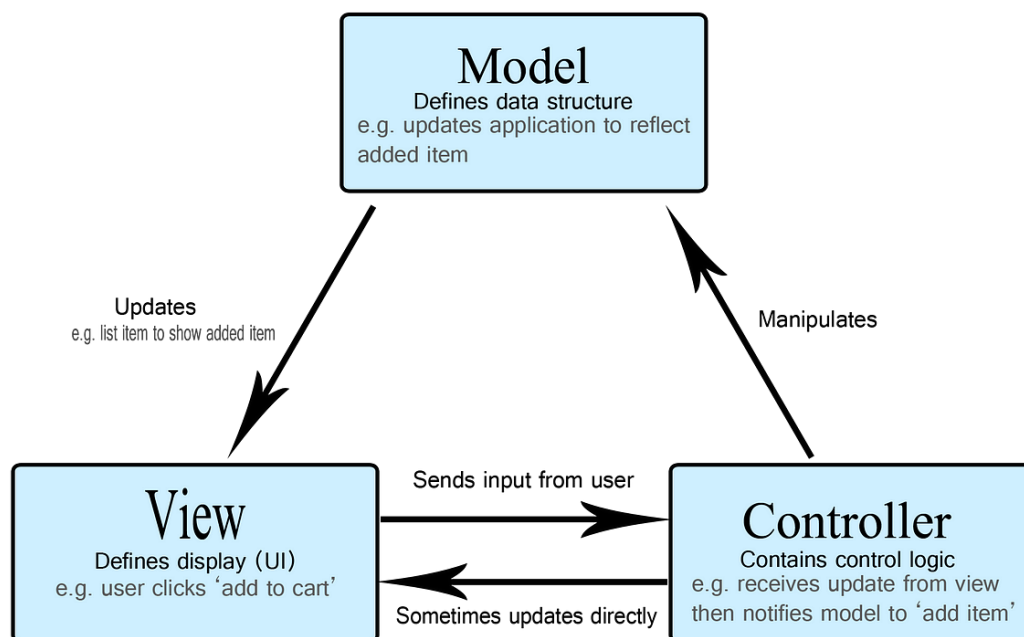
Model-View-Controller (MVC)

Understanding MVC

MVC (Model-View-Controller) is a software architectural pattern that divides an application into three interconnected parts to promote separation of concerns:

- **Model:** Manages the core logic and data of the application.
- **View:** Handles the presentation layer and displays the data.
- **Controller:** Manages input from users, processes it, and updates both the Model and the View accordingly.

This approach improves the structure of the code, making applications easier to maintain, debug, and scale.



MVC Variants

1. Model-View-Presenter (MVP)

In the MVP architecture:

- The **View** displays the user interface and forwards user interactions to the **Presenter**.
- The **Presenter** contains the business logic, interacts with the **Model**, and directly updates the **View**.
- The View in this pattern remains passive and doesn't contain logic, which enhances testability and reusability.

MVP is often used in environments such as Android apps and Windows Forms, where automatic data binding is limited.

2. Model-View-ViewModel (MVVM)

In MVVM:

- The **View** is connected to the **ViewModel** using data binding, which keeps UI components and data in sync.
- The **ViewModel** exposes data and commands for the View and acts as an intermediary between the View and the Model.
- Unlike MVP, the ViewModel does not manipulate the View directly.

MVVM works best with platforms that support binding mechanisms like WPF, Xamarin, or Angular.

When to Choose MVP or MVVM

Choose MVP when:

- The development environment lacks strong support for data binding (e.g., Android, Windows Forms).
- You want to focus on unit testing and maintain a clear separation of concerns.
- You require explicit control over UI updates and logic.

Choose MVVM when:

- You are using modern frameworks with built-in data binding support.
- You prefer a cleaner, more automated approach to UI updates.
- You want to reduce boilerplate and keep UI logic away from the View.